Reporte de Actividades Práctico-Experimentales Nro. 003

1. Datos Generales

Estudiantes	Fabricio Ruiz Granda Carlos Maria Chuico Juan Veitimilla
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de de 2026 09 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.
- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando

buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.
- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. Materiales y reactivos:

Guia practica

Lista de ejercicios

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

Situación:
Un estacionamiento dispone de 4 espacios. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

Procedimiento

1. Identifique el recurso compartido.
2. Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.
3. Cree un semáforo contador inicializado en 4.
4. Cree un proceso vehiculo(id).
5. El vehículo deberá solicitar un espacio antes de ingresar.
6. Si no existen espacios disponibles, deberá permanecer bloqueado.
7. Una vez que obtenga un espacio, deberá simular su permanencia.
8. Al salir, deberá liberar el espacio mediante signal().
9. Cree 10 procesos vehículo.
10. Analice qué sucede cuando llegan más de 4 vehículos simultáneamente.

Pseudocódigo

```
SEMÁFORO espacios = 4


PROCEDIMIENTO vehiculo(id)
escribir("Vehículo ", id, " llega al estacionamiento") wait(espacios)
    escribir("Vehículo ", id, " ingresa al estacionamiento") estacionar(id)


    esperar_tiempo()
```

```
    escribir("Vehículo ", id, " sale del estacionamiento") signal(espacios)

FIN PROCEDIMIENTO


INICIO

    PARA i = 1 HASTA 10 HACER
        crear_proceso(vehiculo, i) FIN PARA

FIN
```

Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

Ejercicio 2. Cajero automático compartido — Exclusión mutua

Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

Situación:
Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura.

Procedimiento

1. Identifique la variable compartida.
2. Determine qué operaciones pueden ejecutarse simultáneamente.
3. Cree un mutex para proteger el saldo.
4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex.
5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.
7. Cree los tres procesos.
8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.
10. Compare el comportamiento con y sin exclusión mutua.

Pseudocódigo

```
saldo = 1000

MUTEX mutex_saldo = 1

PROCEDIMIENTO retirar(id, cantidad) lock(mutex_saldo)

    escribir("Proceso ", id, " intenta retirar $", cantidad)

    SI saldo >= cantidad ENTONCES saldo =

        saldo - cantidad

        escribir("Retiro realizado") escribir("Saldo actual: $",
        saldo)
```

```
    SINO
        escribir("Saldo insuficiente") FIN SI
    unlock(mutex_saldo) FIN
PROCEDIMIENTO
PROCEDIMIENTO depositar(id, cantidad) lock(mutex_saldo)
    escribir("Proceso ", id, " deposita $", cantidad) saldo = saldo +
    cantidad
    escribir("Saldo actual: $", saldo) unlock(mutex_saldo)
FIN PROCEDIMIENTO

INICIO
    crear_proceso(retirar, 1, 300)
    crear_proceso(retirar, 2, 250)
    crear_proceso(depositar, 3, 200) FIN
```

Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:

Una fábrica tiene una línea de producción con capacidad para almacenar **5 piezas** temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

Procedimiento

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.
5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.
14. Debe retirar la pieza.
15. Debe liberar el mutex.

16. Debe indicar que existe un espacio disponible.

Pseudocódigo

```
COLA            almacen
CAPACIDAD = 5

SEMÁFORO espacios = 5 SEMÁFORO
elementos = 0

MUTEX mutex_almacen = 1


PROCEDIMIENTO productor(id)

        REPETIR 5 VECES


      pieza = fabricar_pieza() wait(espacios)
      lock(mutex_almacen) insertar(almacen,
      pieza)
      escribir("Productor ", id,
            " agregó pieza ", pieza)
      unlock(mutex_almacen)
      signal(elementos)
   FIN REPETIR


     FIN PROCEDIMIENTO

PROCEDIMIENTO consumidor(id)

        REPETIR 5 VECES


      wait(elementos)
      lock(mutex_almacen) pieza = extraer(almacen)
      escribir("Consumidor ", id, " retiró
            pieza ", pieza)


      unlock(mutex_almacen)
      signal(espacios) ensamblar(pieza)
   FIN REPETIR

FIN PROCEDIMIENTO


INICIO

   PARA i = 1 HASTA 2 HACER
      crear_proceso(productor, i) FIN PARA

   PARA i = 1 HASTA 2 HACER
      crear_proceso(consumidor, i) FIN PARA

FIN
```

Pregunta de análisis:

¿Qué sucede si espacios se inicializa en 0? ¿Qué sucede si elementos se inicializa en 5?

spaces = 0

Ningún productor obtiene permiso para insertar, aunque el almacén esté vacío y los consumidores también esperen piezas; ambos grupos quedan bloqueados y el trabajo no puede avanzar. En esta implementación, la

inicialización incorrecta se detecta antes de crear los hilos para evitar dejar bloqueada la aplicación.

elements = 5

Los consumidores reciben permisos que afirman que ya hay piezas, aunque la cola comienza vacía; si se retiraran esas piezas, produciría una extracción inválida y los contadores dejarían de representar el estado real de la cola. La implementación también detecta si está inicializada y muestra un diagnóstico sin iniciar los hilos.

Por eso, para representar correctamente un almacén inicialmente vacío, los valores deben ser **spaces = 5** y **elements = 0**.

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación:

Un puente permite circular en ambos sentidos, pero solamente puede existir **una dirección de circulación activa a la vez**.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Procedimiento

1. Identifique el recurso compartido.
2. Defina una variable que indique la dirección actual.
3. Mantenga un contador de vehículos que están cruzando.
4. Proteja las variables mediante un mutex.
5. Un vehículo deberá verificar la dirección antes de entrar.
6. Si existen vehículos en dirección contraria, deberá esperar.
7. Si puede entrar, incrementará el contador correspondiente.
8. Después de cruzar, disminuirá el contador.
9. Cuando no queden vehículos, el puente podrá cambiar de dirección.
10. Analice el problema de inanición.

Pseudocódigo

```
MUTEX mutex = 1
direccion = NINGUNA vehiculos_norte = 0
vehiculos_sur = 0


PROCEDIMIENTO vehiculo_norte(id) lock(mutex)

    MIENTRAS direccion == SUR HACER esperar()
    FIN MIENTRAS

    direccion = NORTE

    vehiculos_norte = vehiculos_norte + 1 unlock(mutex)


    escribir("Vehículo ", id,
             " cruza hacia el NORTE")

    cruzar_puente(id)

    lock(mutex)

    vehiculos_norte = vehiculos_norte - 1
```

```
    SI vehiculos_norte == 0 ENTONCES direccion =
        NINGUNA
    FIN SI

    unlock(mutex)

FIN PROCEDIMIENTO
```

El estudiante deberá completar el procedimiento:
PROCEDIMIENTO vehiculo_sur(id)
siguiendo la misma lógica.
Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

Ejercicio 5. Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:
Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente:
DISEÑO → REVISIÓN → IMPRESIÓN
No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Procedimiento

1. Identifique los procesos participantes.
2. Cree un semáforo para cada etapa.
3. Permita inicialmente que el proceso de diseño pueda ejecutarse.
4. Diseño deberá habilitar a revisión.
5. Revisión deberá habilitar a impresión.
6. Impresión deberá habilitar nuevamente a diseño.
7. Repita el ciclo para 3 documentos.
8. Verifique que el orden siempre sea correcto.
9. Analice qué ocurre si se elimina uno de los signal().

Pseudocódigo

```
SEMÁFORO turno_diseno = 1 SEMÁFORO
turno_revision = 0 SEMÁFORO
turno_impresion = 0

PROCEDIMIENTO diseno()

        REPETIR 3 VECES

      wait(turno_diseno)

      escribir("Diseño prepara documento")

      crear_documento() signal(turno_revision)

  FIN REPETIR

    FIN PROCEDIMIENTO
```

```
PROCEDIMIENTO revision()
        REPETIR 3 VECES
        wait(turno_revision)
        escribir("Revisión verifica documento") revisar_documento()
        signal(turno_impresion)
    FIN REPETIR
FIN PROCEDIMIENTO

PROCEDIMIENTO impresion()
        REPETIR 3 VECES
        wait(turno_impresion)
        escribir("Impresión imprime documento")
        imprimir_documento() signal(turno_diseno)
    FIN REPETIR
FIN PROCEDIMIENTO

INICIO
    crear_proceso(diseno) crear_proceso(revision)
    crear_proceso(impresion)
FIN
```

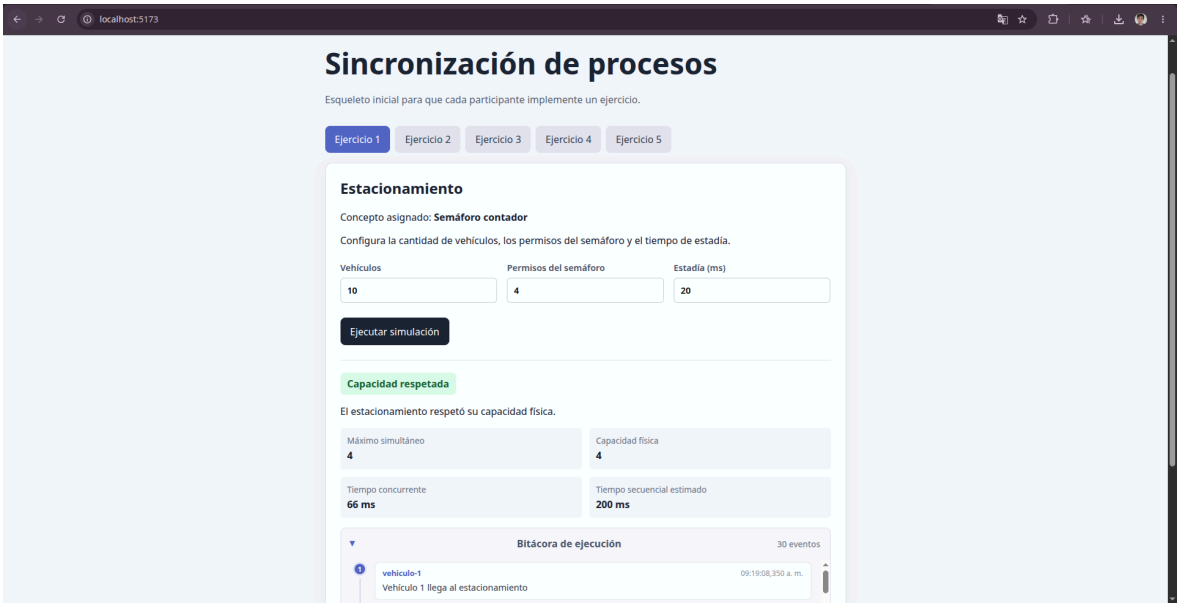
6. Resultados esperados:

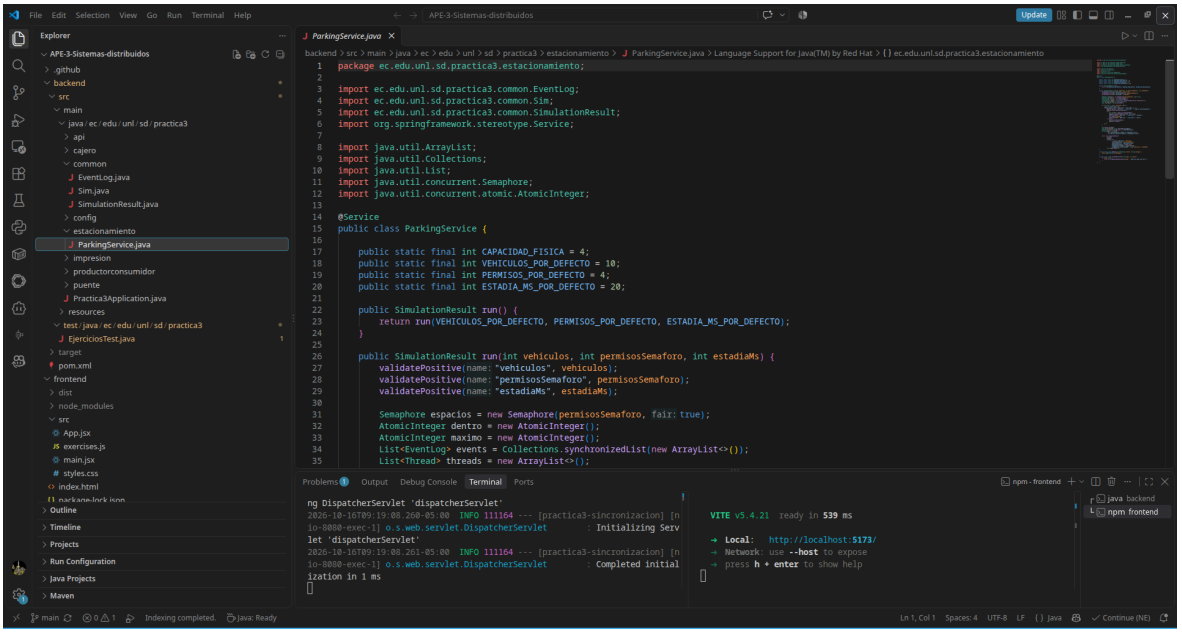
- Comprobación de las distintas formas de como los hilos se comunican.
- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes.
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la práctica

Ejercicio	Resultado Esperado	Criterio de Verificación
1. Estacionamiento	Vehículos dentro ≤ 4	Verificar que nunca ingresen más de 4 simultáneamente
2. Cajero	Saldo final = **$650**	Ejecutar varias veces y comprobar que el saldo no varíe

3. Productor-Consumidor	Todos los productos producidos son consumidos	Verificar que el almacén nunca supere 5 ni quede con inconsistencias
4. Puente	Solo una dirección activa a la vez	Comprobar que nunca crucen vehículos en sentidos opuestos simultáneamente
5. Centro de Impresión	Diseño → Revisión → Impresión	Verificar que ningún proceso avance a la siguiente etapa antes de tiempo

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador





Metodología

Se identificó el estacionamiento como un recurso compartido con capacidad para 4 vehículos y se creó un semáforo contador espacios inicializado en 4. Cada vehículo

se ejecutó como un hilo que solicita un permiso con acquire(), registra su entrada y simula su estadía. Al salir, actualiza el contador dentro y libera el permiso con release(), usando finally para garantizar estos pasos. Finalmente, se ejecutaron 10 hilos y se midió el tiempo total, verificando que nunca se superara la capacidad máxima del estacionamiento.

Análisis de resultados

El resultado cumple el criterio de la guía: nunca hubo más de 4 vehículos dentro al mismo tiempo. El tiempo concurrente (66 ms) coincide con lo esperado, porque los 10 vehículos entran en 3 tandas de 20 ms (4 + 4 + 2). El tiempo secuencial es una estimación. Para obtener una medición real se ejecutó la simulación con 1 permiso: [completar con el tiempo medido].

Cuando llegan más de 4 vehículos a la vez, los primeros 4 obtienen permiso y los demás se bloquean en acquire(). Cada salida libera un permiso y deja entrar a un vehículo en espera. El semáforo mantiene el límite sin que el programador tenga que controlar manualmente quién entra. Además, el semáforo es justo, por lo que los vehículos entran en orden de llegada.

Pregunta de análisis: ¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

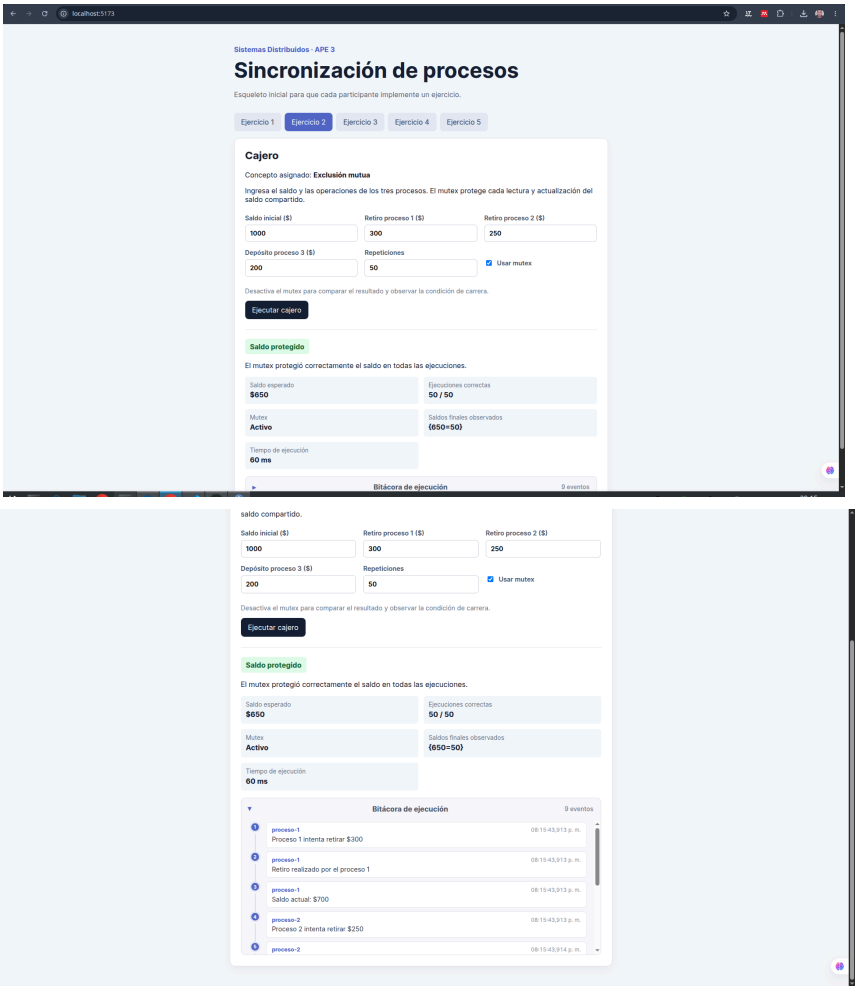
Semáforo en 1: solo entra un vehículo a la vez, así que el semáforo funciona como un mutex. No hay riesgo de superar la capacidad, pero se desperdician 3 de los 4 espacios. El tiempo total se vuelve casi secuencial (cerca de 200 ms). El sistema es seguro pero ineficiente.

Semáforo en 10: los 10 vehículos entran sin esperar, porque hay permisos para todos. El máximo simultáneo llega a completar 10 y supera la capacidad física de 4. El sistema reporta "El semáforo permitió superar la capacidad física". El tiempo baja a cerca de 20 ms, pero a costa de violar la restricción del recurso.

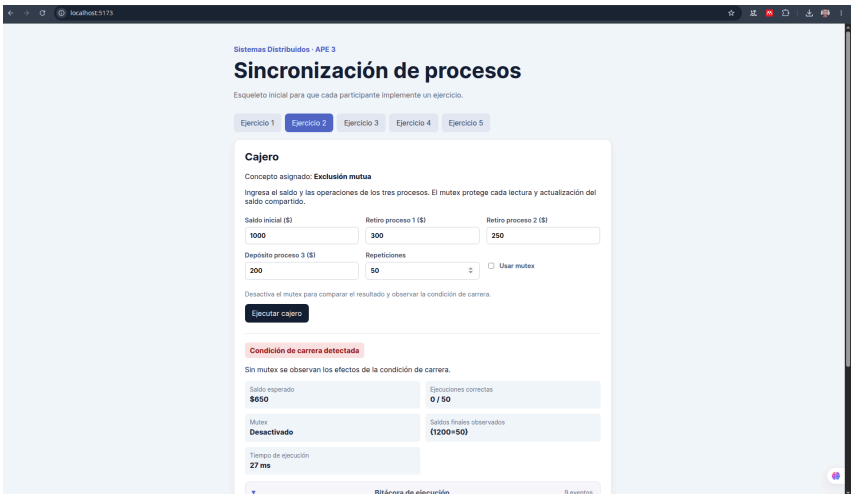
Por eso el valor inicial del semáforo debe coincidir con el número real de recursos disponibles (4).

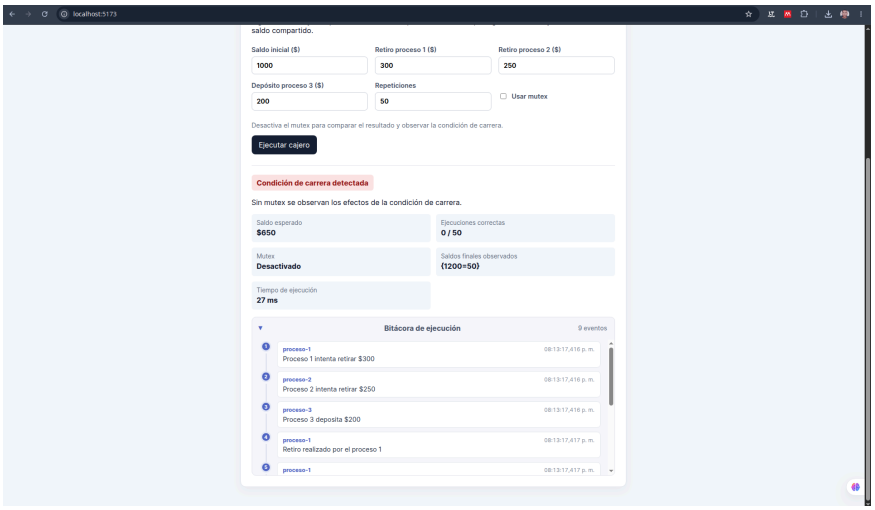
Ejercicio 2. Cajero automático compartido — Exclusión mutua

Con Mutex



Sin Mutex.





Metodología.

Para resolver este ejercicio, primero se identificó el saldo bancario como una variable compartida por los tres hilos. Después, se hace un mecanismo de exclusión mutua utilizando ReentrantLock para el control del acceso al saldo. Antes de realizar cada retiro o depósito, el hilo debe tomar el bloqueo, ejecutar la operación y liberar el bloqueo al finalizar. Al final, se ejecutó el programa para comprobar que las operaciones se realizaron correctamente y que el saldo final era el esperado.

Análisis de resultados:

Al ejecutar el programa, se puede ver que los tres hilos pueden realizar sus operaciones sobre la misma cuenta sin modificar de forma equivocada el saldo. Esto se da porque la exclusión mutua permite que solo un hilo acceda a la sección crítica a la vez. Además, al hacer las diferentes operaciones, se ve que se evita que las operaciones interfieran entre sí y se mantiene la información.

Pregunta de análisis:

¿Por qué el acceso al saldo debe protegerse con un mutex?

Este debe estar protegido porque es una variable compartida por tres hilos, de los cuales dos realizan retiros de 300 y 250 dólares y un tercero de 200 dólares. Si esta no se protege con un mutex, ocurriría una condición de carrera, ya que varios hilos podrían leer el mismo saldo y sobrescribir los cambios realizados por otros.

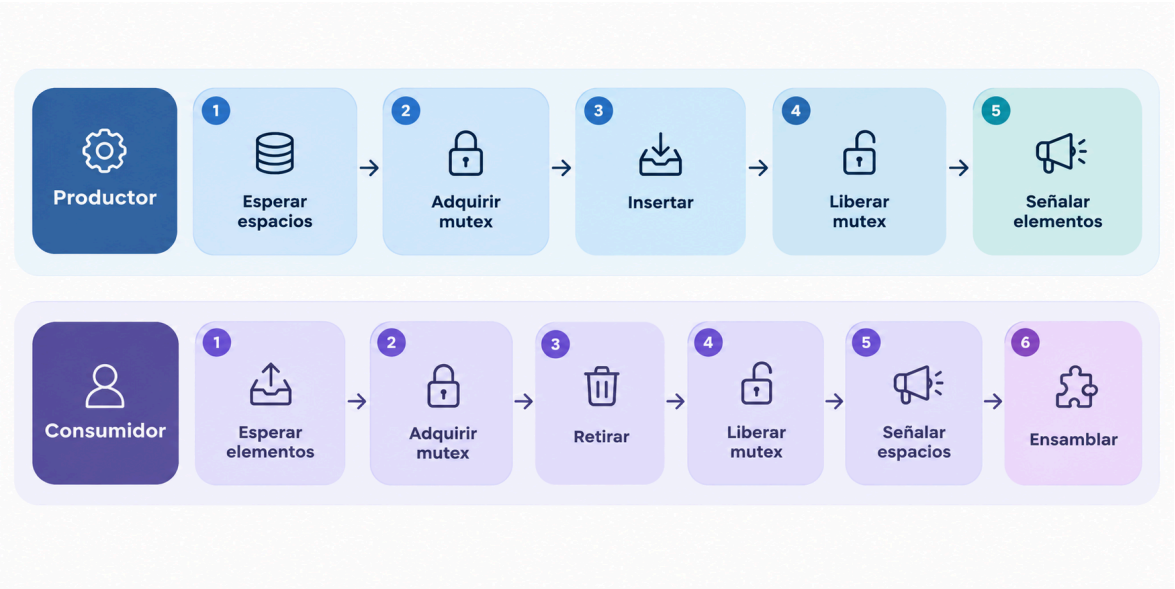
Para que no pase esto, se usa ReentrantLock, que permite controlar el acceso a la variable compartida.

Y también este protege los retiros y los depósitos, así, aunque los hilos se ejecuten simultáneamente, el saldo final siempre será el mismo: 650 dólares.

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor
Metodología / procedimiento

1. Se creó la cola compartida con una capacidad máxima de cinco piezas.
2. Se inicializó **spaces** en 5 porque el almacén comienza vacío y tiene cinco lugares disponibles; **elements** se inicializó en 0 porque todavía no hay piezas para consumir.
3. Se creó un mutex para que solo un hilo a la vez pudiera modificar la cola.
4. Cada productor espera un permiso de **spaces**. Después, adquiere el mutex, inserta una pieza identificada por productor y número de pieza, libera el mutex. Finalmente, incrementa **elements** para avisar que hay una pieza nueva.
5. Cada consumidor espera un permiso de **elements**. Después adquiere el mutex, retira una pieza de la cola y libera el mutex. Finalmente, incrementa spaces para avisar que quedó un lugar disponible y registrar el ensamblaje.
6. Se esperó la finalización de todos los hilos y se verificó que el máximo almacenado no excediera cinco y que la cola terminara vacía.

El orden de sincronización es:



Resultados obtenidos

Indicador	**Resultado esperado y verificado**
Capacidad del almacén	5 piezas
Productores - consumidores	2 - 2
Piezas producidas	10
Piezas consumidas	10
Máximo de piezas almacenadas	no supera 5
Piezas restantes al finalizar	0

Estos resultados cumplen el criterio de la guía: todos los productos son consumidos, el almacén nunca supera su capacidad y queda sin inconsistencias. El tiempo

concurrente se mide en cada ejecución y puede variar según el equipo y la planificación de los hilos.

Pregunta de análisis:

¿Qué sucede si espacios se inicializa en 0? ¿Qué sucede si elementos se inicializa en 5?

spaces = 0

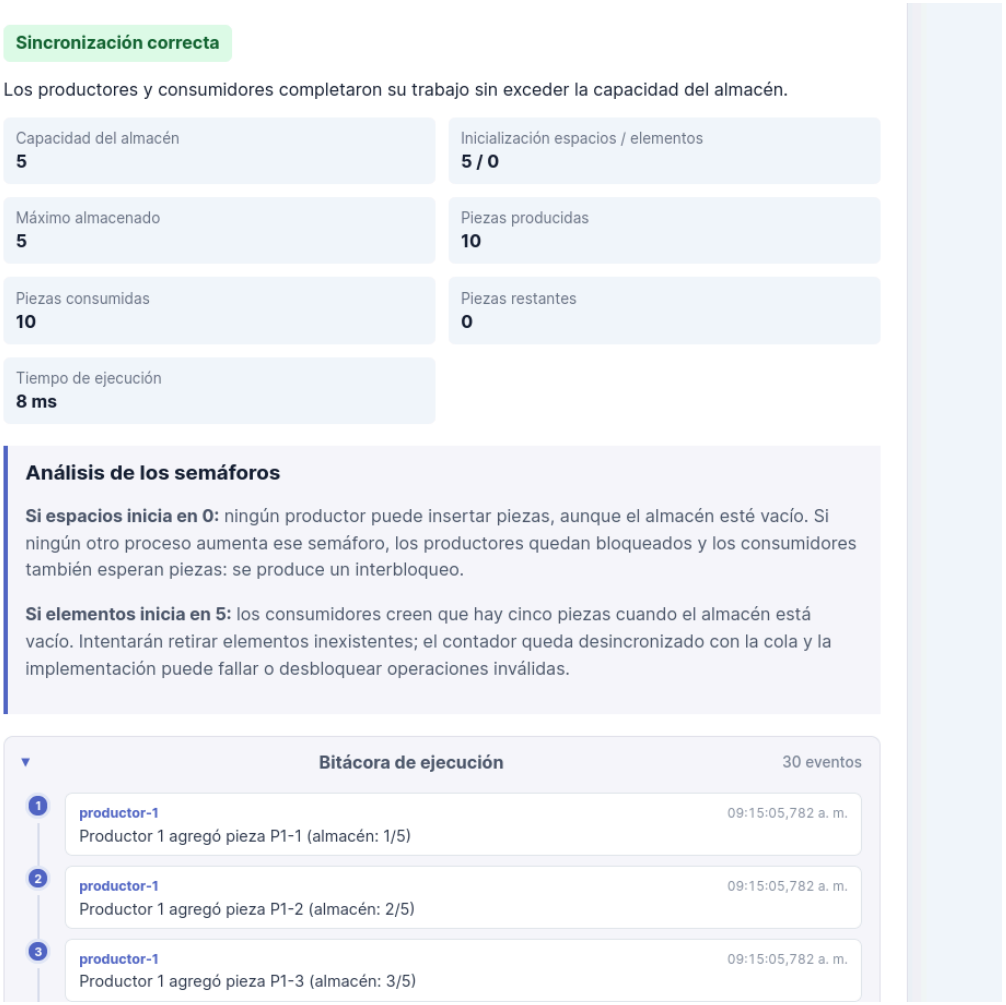
Ningún productor obtiene permiso para insertar, aunque el almacén esté vacío y los consumidores también esperen piezas; ambos grupos quedan bloqueados y el trabajo no puede avanzar. En esta implementación, la inicialización incorrecta se detecta antes de crear los hilos para evitar dejar bloqueada la aplicación.

elements = 5

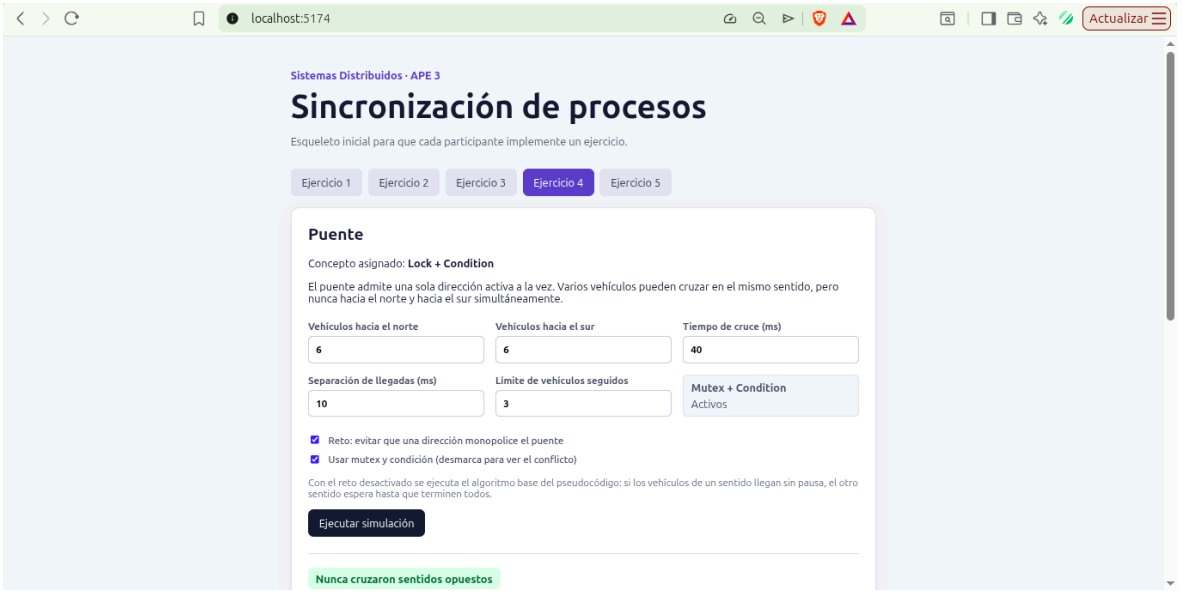
Los consumidores reciben permisos que afirman que ya hay piezas, aunque la cola comienza vacía; si se retiraran esas piezas, produciría una extracción inválida y los contadores dejarían de representar el estado real de la cola. La implementación también detecta si está inicializada y muestra un diagnóstico sin iniciar los hilos.

Por eso, para representar correctamente un almacén inicialmente vacío, los valores deben ser **spaces = 5** y **elements = 0**.

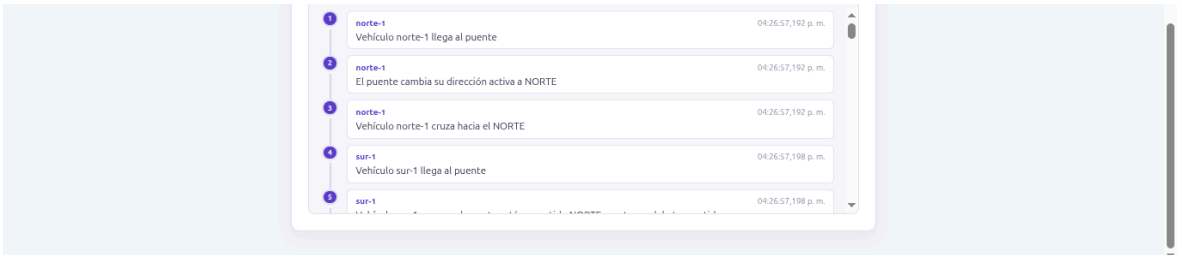


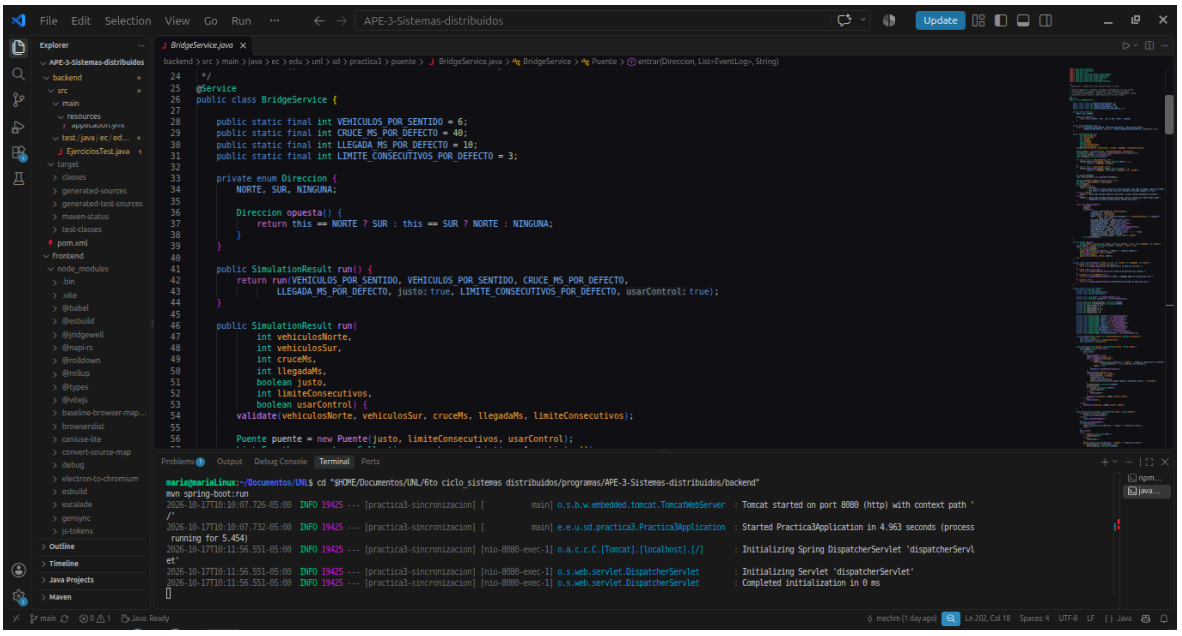


Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

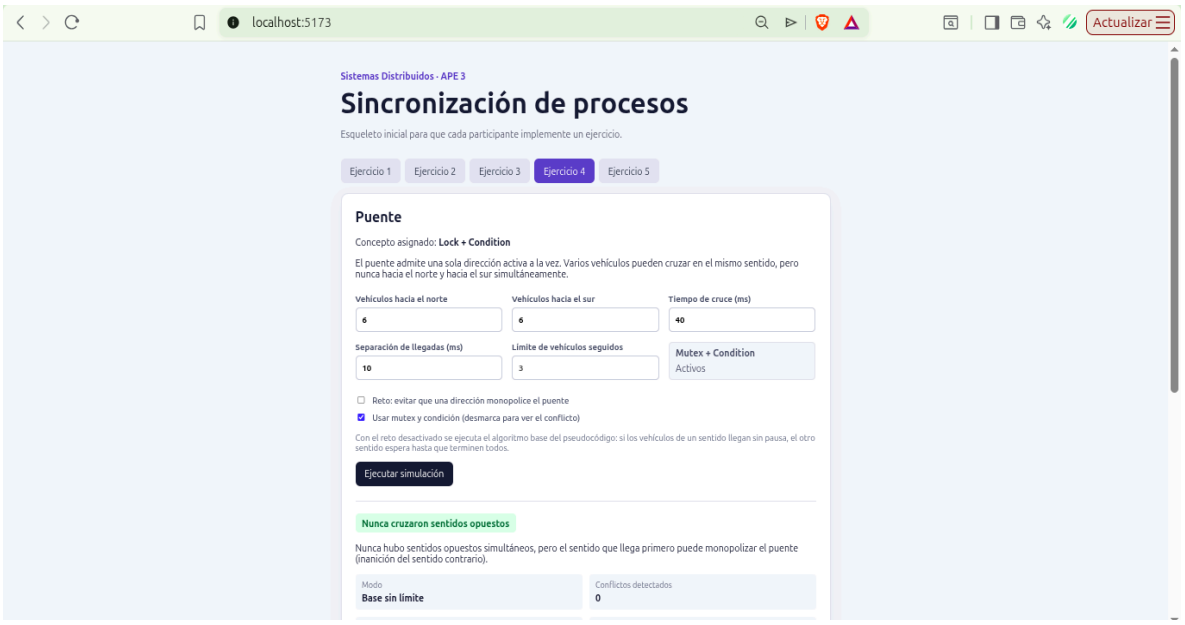




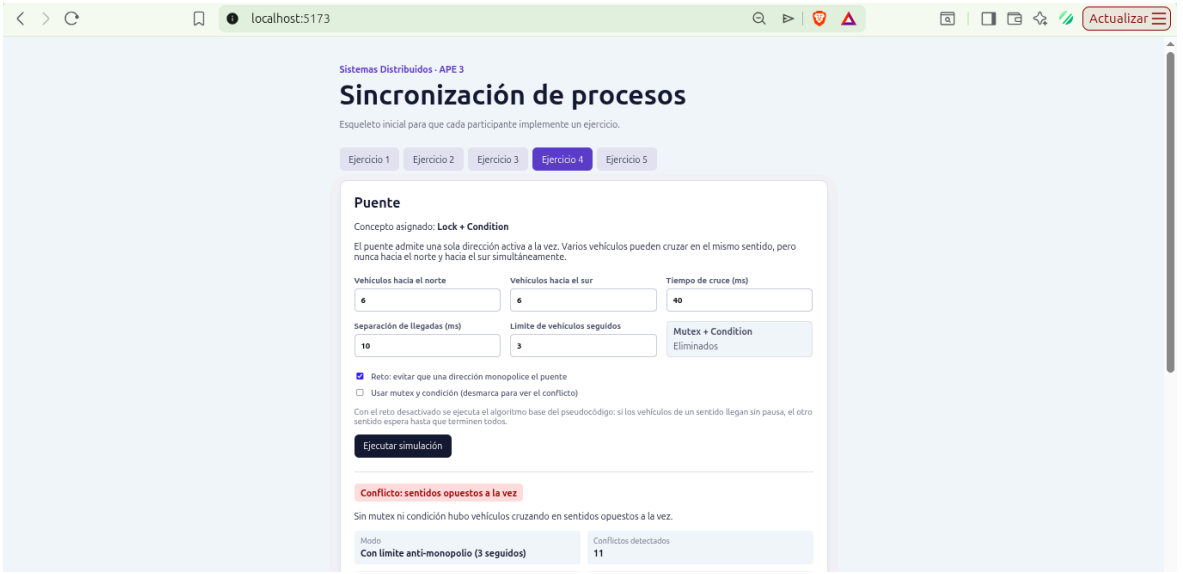


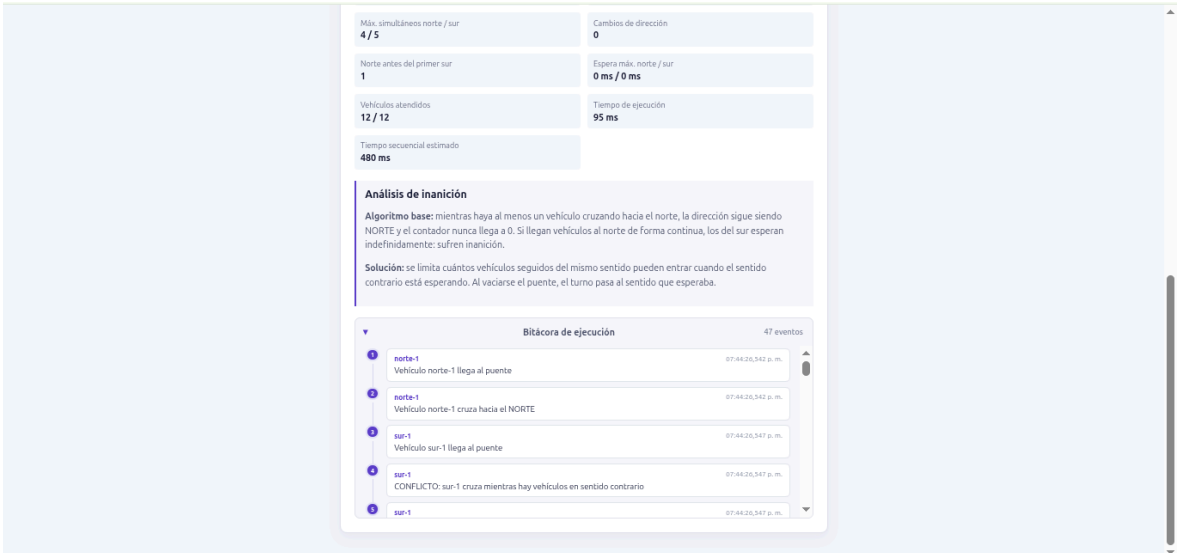


Inanición: pasan los 6 del norte primero



Sin mutex: Aparecen conflictos y el veredicto sale en rojo





Metodología

Para la realización de este ejercicio se identificó primero el punto como recurso compartido por todos los vehículos. Se definió la variable `direccionActual` que

indica el sentido permitido (NORTE, SUR o NINGUNA) y dos contadores, `vehiculosNorte` y `vehiculosSur` que registran cuántos vehículos están cruzando en cada sentido. Después, se implementó mutex con `ReentrantLock` para proteger estas variables, junto con una variable de condición (`Condition`) para que los vehículos esperen. Cada vehículo toma el bloqueo, verifica la dirección y espera si hay vehículos en sentido contrario (puede entrar, incrementa su contador y cruza). Al salir disminuye el contador y cuando el puente queda vacío avisa a los que esperan. Al final, se ejecutó el programa varias veces con y sin mutex para comprobar que nunca cruzaron vehículos de sentidos opuestos a la vez.

Análisis de resultados

Con el mutex se garantizó que nunca cruzaran sentidos opuestos a la vez pero el algoritmo base dado permitió que el norte se quedará con el puente. Al aplicar el reto, que limita a tres los vehículos seguidos del mismo sentido el turno se alterna entre ambos sentidos sin conflictos. Finalmente, al desactivar el mutex aparecieron conflictos, lo que confirma que la exclusión mutua es importante.

Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

```
MUTEX mutex = 1
CONDICION puede_entrar
CONSTANTE LIMITE = 3

direccionActual = NINGUNA
vehiculosNorte = 0
vehiculosSur = 0
esperandoNorte = 0
esperandoSur = 0
consecutivos = 0
turnoPreferente = NINGUNA

FUNCION puedeEntrarAhora(sentido)        // se llama con el mutex tomado
  SI direccionActual == sentido ENTONCES
    // mismo sentido: entra, salvo que ya pasaron LIMITE seguidos
    // y el sentido contrario está esperando
    RETORNAR NO (consecutivos >= LIMITE Y esperando_del_opuesto(sentido) > 0)
  FIN SI

  SI direccionActual == NINGUNA ENTONCES
    // puente libre: entra si nadie tiene preferencia o la tiene este sentido
    RETORNAR turnoPreferente == NINGUNA O turnoPreferente == sentido
  FIN SI

  RETORNAR FALSO                  // hay vehículos en sentido contrario
FIN FUNCION

PROCEDIMIENTO vehiculo_norte(id)
  lock(mutex)

  esperandoNorte = esperandoNorte + 1
  MIENTRAS NO puedeEntrarAhora(NORTE) HACER
    esperar(puede_entrar, mutex)      // libera el mutex mientras espera
  FIN MIENTRAS
  esperandoNorte = esperandoNorte - 1

  SI direccionActual != NORTE ENTONCES
    direccionActual = NORTE
    consecutivos = 0
```

```
    FIN SI
    turnoPreferente = NINGUNA
    consecutivos = consecutivos + 1
    vehiculosNorte = vehiculosNorte + 1

    unlock(mutex)

    escribir("Vehículo ", id, " cruza hacia el NORTE")
    cruzar_puente(id)

    lock(mutex)

    vehiculosNorte = vehiculosNorte - 1

    SI vehiculosNorte + vehiculosSur == 0 ENTONCES
      direccionActual = NINGUNA
      consecutivos = 0
      SI esperandoSur > 0 ENTONCES
        turnoPreferente = SUR        // el sentido contrario tiene el siguiente turno
      FIN SI
      despertar_todos(puede_entrar)
    FIN SI

    unlock(mutex)
FIN PROCEDIMIENTO

PROCEDIMIENTO vehiculo_sur(id)
    lock(mutex)

    esperandoSur = esperandoSur + 1
    MIENTRAS NO puedeEntrarAhora(SUR) HACER
      esperar(puede_entrar, mutex)
    FIN MIENTRAS
    esperandoSur = esperandoSur - 1

    SI direccionActual != SUR ENTONCES
      direccionActual = SUR
      consecutivos = 0
    FIN SI
    turnoPreferente = NINGUNA
    consecutivos = consecutivos + 1
    vehiculosSur = vehiculosSur + 1

    unlock(mutex)

    escribir("Vehículo ", id, " cruza hacia el SUR")
    cruzar_puente(id)

    lock(mutex)

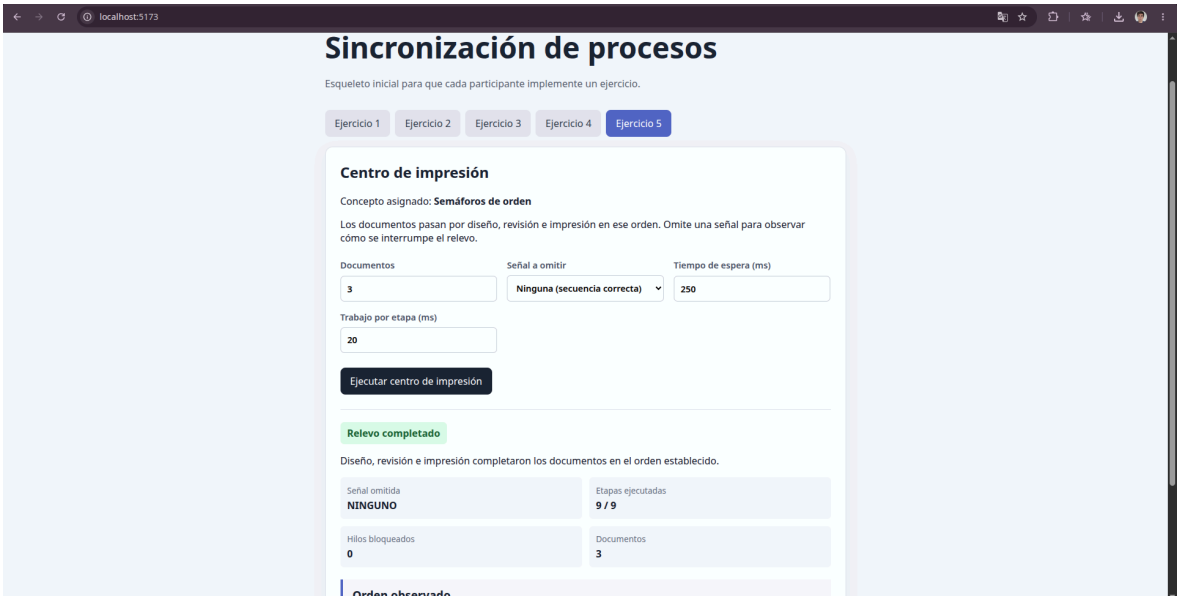
    vehiculosSur = vehiculosSur - 1

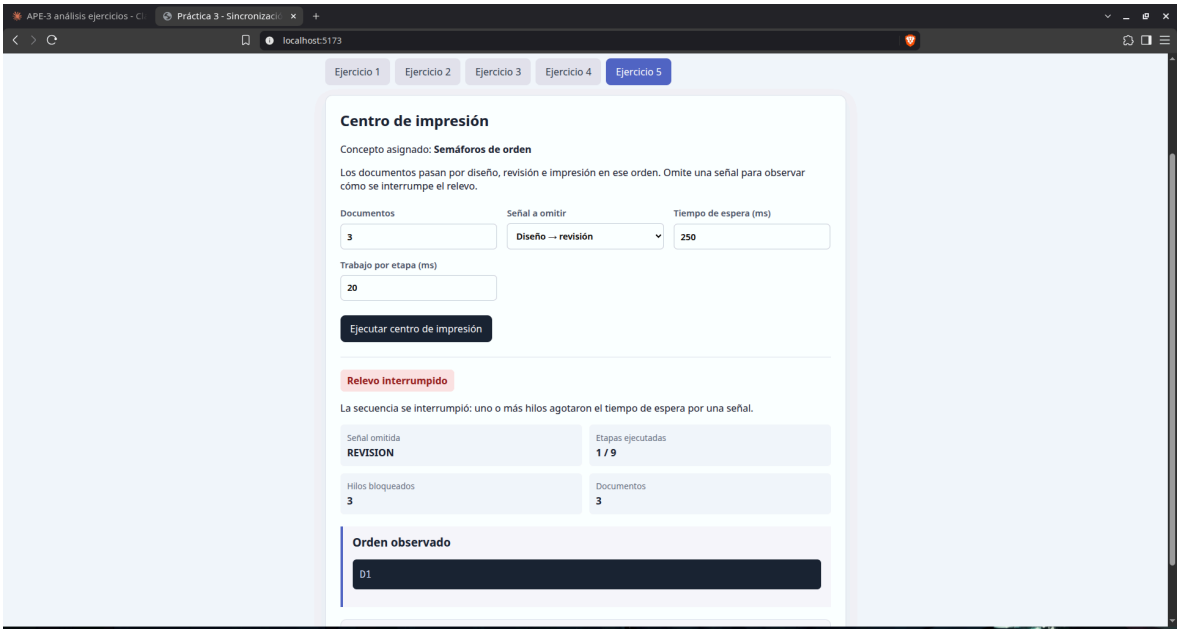
    SI vehiculosNorte + vehiculosSur == 0 ENTONCES
      direccionActual = NINGUNA
      consecutivos = 0
      SI esperandoNorte > 0 ENTONCES
        turnoPreferente = NORTE
      FIN SI
      despertar_todos(puede_entrar)
    FIN SI

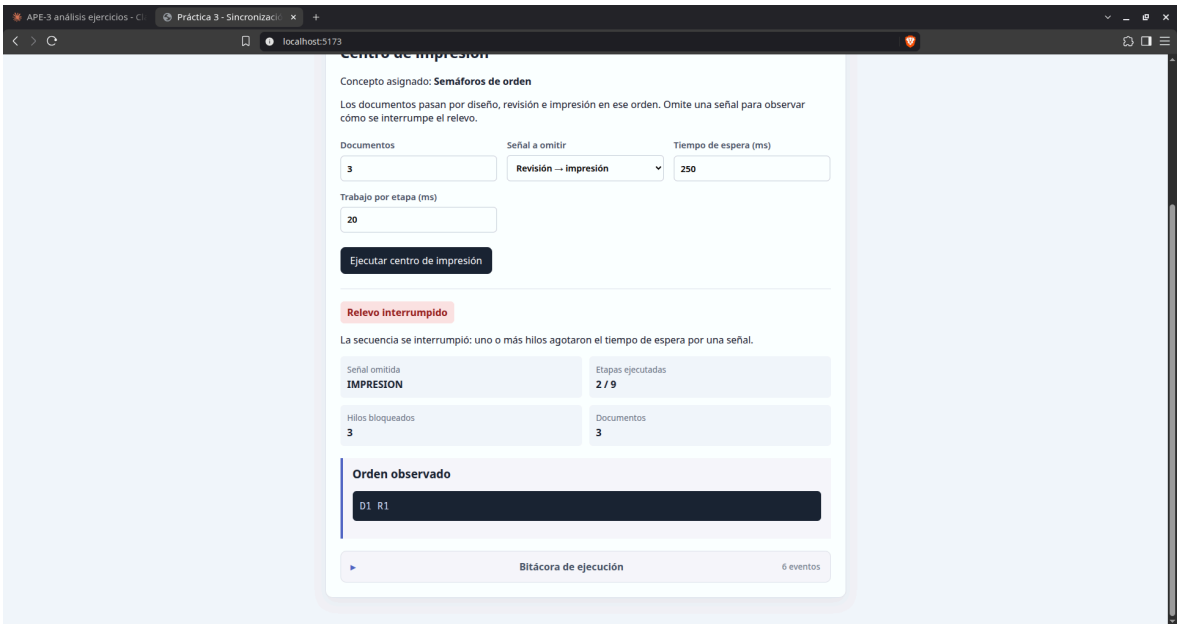
    unlock(mutex)
FIN PROCEDIMIENTO
```

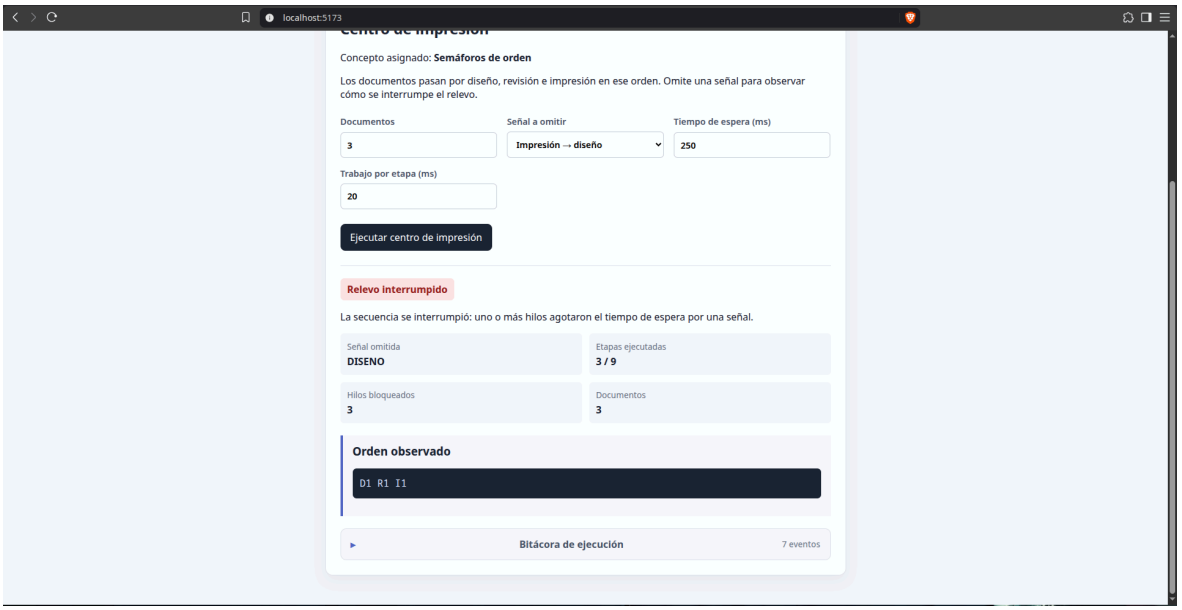
INICIO
  PARA i = 1 HASTA 6 HACER crear_proceso(vehiculo_norte, i) FIN PARA
  PARA i = 1 HASTA 6 HACER crear_proceso(vehiculo_sur, i) FIN PARA
FIN

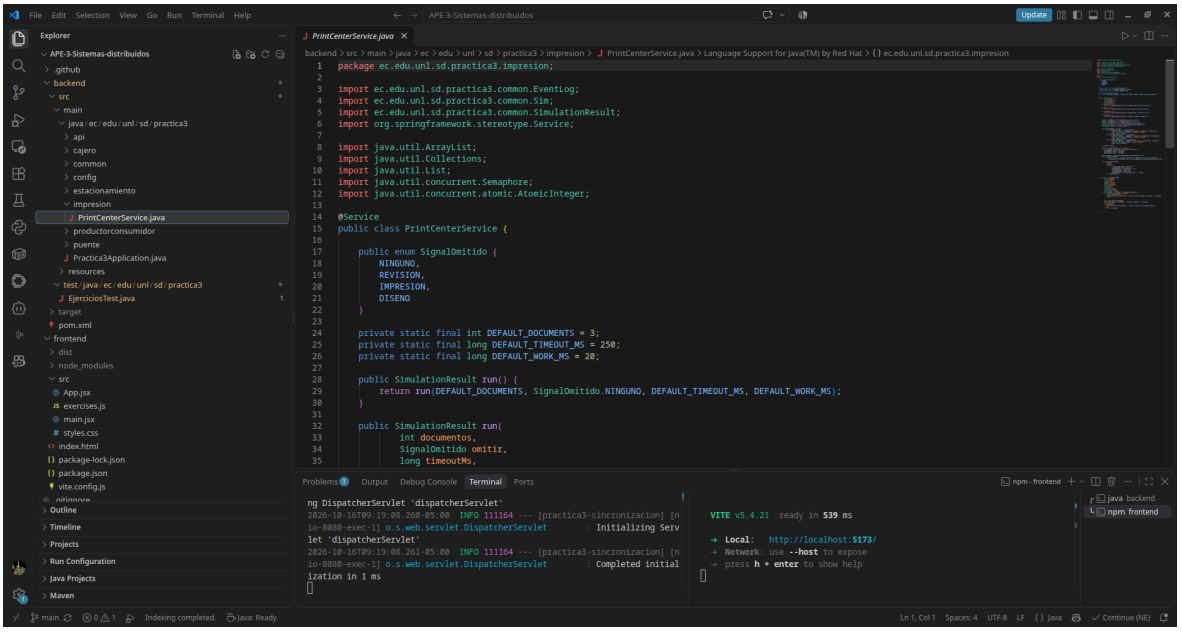
Ejercicio 5. Centro de impresión — Sincronización de procesos











Metodología

Se crearon tres hilos para las etapas de Diseño, Revisión e Impresión, cada uno

encargado de procesar 3 documentos mediante semáforos que controlan el orden de ejecución. Cada etapa realiza su trabajo durante 20 ms y habilita a la siguiente, formando un ciclo. Se utilizó tryAcquire() con un límite de 250 ms para evitar que la aplicación se quede bloqueada. Finalmente, se verificó que el orden fuera el esperado y se añadió la opción de omitir una señal para analizar cómo afecta al funcionamiento del proceso.

Análisis de resultados

Con la secuencia correcta se cumple el criterio de la guía: los tres documentos pasan por Diseño, Revisión e Impresión en ese orden, y ningún proceso avanza a la siguiente etapa antes de tiempo. En ninguna de las ejecuciones aparece una etapa fuera de orden. Cuando se omite una señal, el orden que sí se ejecuta sigue siendo correcto, pero el ciclo se detiene.

El orden se mantiene porque los semáforos funcionan como un testigo que pasa de una etapa a otra, y solo una etapa lo tiene en cada momento. Por eso el orden observado es el mismo en todas las ejecuciones. Esto implica también que el ciclo no tiene paralelismo: cada etapa espera a la anterior, así que el tiempo concurrente es prácticamente igual al secuencial. El objetivo del ejercicio es garantizar el orden de ejecución, no mejorar el rendimiento.

Pregunta de análisis: ¿Qué ocurre si se elimina uno de los signal()?

Al omitir una de las señales, se rompe el ciclo y las etapas quedan esperando su turno. Si se elimina Diseño → Revisión, solo se ejecuta D1; si se elimina Revisión → Impresión, se ejecutan D1 y R1; y si se elimina Impresión → Diseño, se completa el primer documento. El timeout permite detectar el bloqueo y evitar que la aplicación quede detenida. Esto demuestra que cada signal() es necesario para mantener el orden de ejecución y completar el proceso.

7. Preguntas de Control:

- **¿Qué es una condición de carrera?**

  La condición de carrera se produce cuando dos o más hilos acceden al mismo recurso compartido y al menos uno modifica su valor, pudiendo generar resultados inesperados dependiendo del orden de ejecución.

- **¿Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas.**

  La diferencia que existe es que el mutex se usa para proteger un recurso compartido, mientras que el semáforo se usa para controlar la sincronización entre los hilos.

  Estos no se los puede usar en todos los casos ya que tienen algunas diferencias:

  Propiedad: El mutex solo se libera por el hilo que bloqueó, mientras que un semáforo se puede liberar por otro hilo.

  Uso: El mutex se usa para evitar que varios hilos modifiquen al mismo tiempo un recurso que está compartido, mientras que el semáforo binario permite coordinar la ejecución de hilos y indicar cuándo se puede continuar.

- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**
  Deadlock: Esto pasa cuando dos o más hilos quedan bloqueados porque intentan acceder a un recurso que está siendo usado por otro.
  Las condiciones necesarias:
  - Exclusión mutua: Un recurso solo puede ser utilizado por un proceso a la vez.
  - Retención y espera: Un proceso mantiene un recurso mientras espera obtener otro.
  - No apropiación: Un recurso no puede ser retirado por la fuerza a un proceso; este debe liberarlo voluntariamente.
  - Espera circular: Existe una cadena de procesos en la que cada uno espera un recurso que posee el siguiente.

8. Evaluación

Criterio	Excelente	Bueno	Bajo
Funcionamiento (back-end, front-end) se evaluara criterios de Verificacion	Funciona completo (4)	Parcial (2)	No funciona (0)
Documento de practica (Enlace a git, metodología usada por el alumno, resultados, preguntas de control, bibliografía, conclusiones, video de explicación)	Funciona completo (3)	Parcial (2)	No funciona (0)
Defensa de practica	Todos los puntos (3)	Parcial (1)	No valido (0)

9. Bibliografía

Semana 3. Guaman, Jose. 2026

10. Anexos

https://github.com/Carlosjosu/APE-3-Sistemas-distribuidos

11. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en Firmadi Firmado electrónicamente por: **JOSE OSWALDO GUAMAN QUINCHE**
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en Firmadi Firmado electrónicamente por: **RAMIRO RENE RIVERA GUAMAN**
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC 1859